CANTEEN FOOD WASTE PREDICTION & TRACKER

Using Arrays, Sorting and Searching

IT23302 – DATA STRUCTURES PROJECT

PROJECT MEMBERS	REGISTER NUMBER
NAVEENA	2025115047
HARNI	2025115103

Submitted as a Data Structures course project

Institution / Department: _____

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project report entitled “Canteen Food Waste Prediction & Tracker” is submitted by NAVEENA (2025115047) and HARNI (2025115103) as part of the IT23302 – Data Structures Project. The work presented in this report is based on the project implementation submitted by the students.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

DECLARATION

We declare that this project report titled “Canteen Food Waste Prediction & Tracker” has been prepared for the IT23302 – Data Structures Project. The report describes the design and implementation of the C program submitted with the project.

NAVEENA – 2025115047 _____

HARNI – 2025115103 _____

ACKNOWLEDGEMENT

We express our sincere thanks to our course faculty and project guide for their guidance and support throughout this project. We also thank our department and institution for providing the learning environment and resources required to complete this Data Structures project.

TABLE OF CONTENTS

ABSTRACT

1. INTRODUCTION

2. AIM AND OBJECTIVES

3. PROBLEM STATEMENT

4. SCOPE OF THE PROJECT

5. SOFTWARE AND HARDWARE REQUIREMENTS

6. DATA STRUCTURES AND CONCEPTS USED

7. SYSTEM DESIGN AND PROGRAM FLOW

8. ALGORITHMS

9. MODULE DESCRIPTION

10. IMPLEMENTATION DETAILS

11. TESTING AND EXPECTED RESULTS

12. ADVANTAGES

13. LIMITATIONS

14. FUTURE ENHANCEMENTS

15. CONCLUSION

REFERENCES

APPENDIX A – C PROGRAM SOURCE CODE

ABSTRACT

This project, titled “Canteen Food Waste Prediction & Tracker”, is a menu-driven C program designed to record food preparation and consumption details in a canteen. It calculates the quantity and percentage of food wasted, displays a report, ranks records by the highest waste, searches records by item name, and estimates the quantity to prepare for the next day using previous consumption data. The program uses an array of structures to store records, Bubble Sort to arrange records by waste quantity, Linear Search to find an item, and file handling to preserve records between program runs. The project demonstrates how fundamental data-structure concepts can be applied to a practical canteen-management problem.

1. INTRODUCTION

Food waste is a common problem in canteens. When the quantity of food prepared is not compared regularly with the quantity consumed, staff may repeatedly prepare more food than required. This can increase operating costs and lead to unnecessary waste.

The Canteen Food Waste Prediction & Tracker provides a simple way to maintain daily records. For each food item, the user enters the item name, date, quantity prepared and quantity consumed. The application calculates the waste automatically and provides options to view, sort, search and predict quantities.

The project is implemented in the C programming language and focuses on arrays, structures, sorting, searching and file handling.

2. AIM AND OBJECTIVES

Aim: To develop a simple C application that records canteen food quantities, identifies waste and supports better preparation planning.

Objectives:

• Maintain records of food items, dates, prepared quantities and consumed quantities.

• Calculate the quantity and percentage of food wasted.

• Display a report containing all stored records.

• Sort records in descending order of wasted quantity.

• Search for records using the food item name.

• Estimate the next day's preparation quantity from the average past consumption.

• Save records in a text file and load them when the application starts.

3. PROBLEM STATEMENT

Canteen staff may not have a systematic record of how much food is prepared, consumed and left over. Without historical information, it is difficult to identify frequently wasted items or estimate suitable preparation quantities. A lightweight record-management application is therefore required to store daily information and present useful summaries.

4. SCOPE OF THE PROJECT

The current implementation supports up to 100 records in an array. It accepts one-word item names and dates in DD-MM-YYYY format. It calculates waste, displays records, sorts by waste, searches by item name, predicts a suggested quantity and stores records in the local text file canteen_data.txt.

The present source code implements Linear Search by item name and Bubble Sort by waste quantity. Date-based sorting and Binary Search are described as possible data-structure concepts in the initial project outline, but they are not implemented in the supplied program code.

5. SOFTWARE AND HARDWARE REQUIREMENTS

Software requirements:

• Programming language: C

• C compiler: GCC or another standard C compiler

• Development environment: Any C-compatible IDE or text editor

• Operating system: Windows, Linux or macOS

Hardware requirements:

• A computer capable of running a C compiler

• Keyboard and display

• Sufficient storage for the executable and canteen_data.txt data file

6. DATA STRUCTURES AND CONCEPTS USED

6.1 Structure: The FoodRecord structure groups the item name, date, prepared quantity, consumed quantity, wasted quantity and waste percentage into one record.

6.2 Array of structures: A global array named records stores up to MAX (100) FoodRecord entries. The variable count tracks the number of records currently stored.

6.3 Sorting: Bubble Sort arranges records in descending order of wasted quantity. Adjacent records are swapped whenever the earlier record has less waste than the next record.

6.4 Searching: Linear Search checks each stored record and compares its item name with the requested name using strcmp().

6.5 File handling: saveToFile() writes records to canteen_data.txt, while loadFromFile() reads the saved records when the program starts.

7. SYSTEM DESIGN AND PROGRAM FLOW

1. Start the application.

2. Load previously saved records from canteen_data.txt, if the file exists.

3. Display the menu and read the user's choice.

4. Add Record: read item details, calculate waste and append the record to the array.

5. Show Report: display all records and their calculated values.

6. Sort: arrange records by highest waste first.

7. Search: find and display records matching an item name.

8. Predict: calculate average consumption for the requested item and add a buffer of five units.

9. Save & Exit: write all current records to the text file and terminate.

10. Repeat menu operations until the user selects Save & Exit.

8. ALGORITHMS

8.1 Waste calculation: wasted = prepared − consumed. If prepared is greater than zero, waste percentage = (wasted / prepared) × 100; otherwise, the percentage is set to zero.

8.2 Prediction: For all records matching an item name, calculate total consumed and number of matching records. Integer average consumption = total consumed / number of matching records. Suggested quantity = average consumption + 5 units.

8.3 Bubble Sort: Compare adjacent records from left to right. Swap them when the left record has a smaller wasted quantity. Repeat passes until the array is ordered from highest waste to lowest waste.

8.4 Linear Search: Start from the first record. Compare each item name with the search key. Print every matching record; if no match is found, display “Record not found!”.

9. MODULE DESCRIPTION

addRecord(): Checks whether the array is full, accepts a new record, calls calculateWaste() and increments count.

calculateWaste(int i): Calculates the wasted quantity and waste percentage for the record at index i.

showReport(): Displays a tabular report of all stored records. If no records exist, it displays a message.

predictQuantity(char *name): Finds matching records, calculates average consumption and displays the suggested preparation quantity.

searchRecord(): Accepts an item name and uses Linear Search to display matching records.

sortRecords(): Applies Bubble Sort in descending order of wasted quantity.

saveToFile(): Writes all records to canteen_data.txt.

loadFromFile(): Loads previously saved records from canteen_data.txt.

main(): Loads saved data and repeatedly displays the menu until the user chooses Save & Exit.

10. IMPLEMENTATION DETAILS

The program uses the standard header files stdio.h, string.h and stdlib.h. The FoodRecord structure is declared with character arrays for the item name and date, integer fields for quantities and a floating-point field for waste percentage.

The menu provides six choices: Add Record, Show Report, Sort by Waste (Highest First), Search Record by Item Name, Predict Next Day Quantity, and Save & Exit. The application loads the data file before showing the menu. When the exit option is selected, it saves the current array and terminates.

Input validation is limited in the supplied implementation. The user should enter non-negative quantities and ensure that consumed quantity does not exceed prepared quantity.

11. TESTING AND EXPECTED RESULTS

The following test cases can be used to verify the program. They are test scenarios derived from the program logic, not claims of an independently executed test run.

Test 1 – Add a record: Enter Rice, 10-11-2026, prepared 100 and consumed 80. Expected calculation: wasted = 20 and waste percentage = 20.00%.

Test 2 – Display report: After adding a record, choose Show Report. Expected result: the item, date, prepared, consumed, wasted and waste percentage are displayed.

Test 3 – Sort: Add multiple records with different waste quantities and choose Sort. Expected result: records are arranged from the highest wasted quantity to the lowest.

Test 4 – Search: Search for an item that exists. Expected result: all matching records are displayed. Search for a missing item. Expected result: “Record not found!”.

Test 5 – Prediction: If an item has consumed quantities 80 and 90 in two records, the integer average is 85 and the suggested quantity is 90 units after adding the five-unit buffer.

Test 6 – Persistence: Choose Save & Exit, then restart the program. Expected result: records previously written to canteen_data.txt are loaded.

12. ADVANTAGES

• Provides a simple and organized way to maintain food records.

• Calculates waste quantity and percentage automatically.

• Highlights high-waste records through sorting.

• Helps estimate preparation quantities from earlier consumption.

• Preserves records between program runs using a text file.

• Demonstrates practical use of basic C data structures and algorithms.

13. LIMITATIONS

• The array can store a maximum of 100 records.

• Item names are read as single words; names containing spaces are not supported by the current scanf() input.

• The prediction uses a simple average and a fixed five-unit buffer; it does not account for weekdays, holidays or changing demand.

• The program does not currently provide date-based sorting or Binary Search.

• Input validation and handling of invalid quantity values can be improved.

• The report is text-based and does not provide graphical charts or a database-backed interface.

14. FUTURE ENHANCEMENTS

• Support item names containing spaces and validate all user inputs.

• Add date-based sorting and Binary Search on date-sorted records.

• Replace the fixed prediction buffer with a configurable or data-driven method.

• Add weekly and monthly summaries, charts and category-wise analysis.

• Increase storage capacity or use a database for larger datasets.

• Provide a graphical user interface and user login for canteen staff.

15. CONCLUSION

The Canteen Food Waste Prediction & Tracker demonstrates how arrays, structures, sorting, searching and file handling can be combined to address a practical problem. It records prepared and consumed quantities, calculates waste, displays reports, ranks records by waste and estimates future preparation quantities from past consumption. Although the current version is a basic command-line application, it provides a foundation that can be extended with stronger validation, date-based operations and more advanced prediction techniques.

REFERENCES

1. IT23302 – Data Structures Project outline and supplied C program source code.

2. Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, 2nd Edition, Prentice Hall.

3. Standard C library documentation for stdio.h, string.h and stdlib.h.

APPENDIX A – C PROGRAM SOURCE CODE

The following is a cleaned, bounded-input version of the supplied program. The core project operations and prediction formula are retained.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 100

struct FoodRecord {
    char itemName[50];
    char date[15];
    int prepared;
    int consumed;
    int wasted;
    float wastePercentage;
};

struct FoodRecord records[MAX];
int count = 0;

void calculateWaste(int i) {
    records[i].wasted = records[i].prepared - records[i].consumed;
    if (records[i].prepared > 0) {
        records[i].wastePercentage =
            ((float)records[i].wasted / records[i].prepared) * 100;
    } else {
        records[i].wastePercentage = 0;
    }
}

void addRecord() {
    if (count >= MAX) {
        printf("Storage is full!\\n");
        return;
    }

    printf("Enter Item Name: ");
    scanf("%49s", records[count].itemName);
    printf("Enter Date (DD-MM-YYYY): ");
    scanf("%14s", records[count].date);
    printf("Enter Quantity Prepared: ");
    scanf("%d", &records[count].prepared);
    printf("Enter Quantity Consumed: ");
    scanf("%d", &records[count].consumed);

    calculateWaste(count);
    count++;
    printf("Record added successfully!\\n");
}

void showReport() {
    if (count == 0) {
        printf("No records found.\\n");
        return;
    }

    printf("\\n--- Canteen Food Waste Report ---\\n");
    printf("%-15s %-12s %-10s %-10s %-10s %-10s\\n",
           "Item", "Date", "Prepared", "Consumed", "Wasted", "Waste %");

    for (int i = 0; i < count; i++) {
        printf("%-15s %-12s %-10d %-10d %-10d %-9.2f%%\\n",
               records[i].itemName, records[i].date,
               records[i].prepared, records[i].consumed,
               records[i].wasted, records[i].wastePercentage);
    }
}

void predictQuantity(char *name) {
    int totalConsumed = 0, matches = 0;

    for (int i = 0; i < count; i++) {
        if (strcmp(records[i].itemName, name) == 0) {
            totalConsumed += records[i].consumed;
            matches++;
        }
    }

    if (matches > 0) {
        int avgConsumed = totalConsumed / matches;
        printf("\\nPrediction for %s: Suggested quantity to prepare "
               "tomorrow is approximately %d units based on past consumption.\\n",
               name, avgConsumed + 5);
    } else {
        printf("Item not found for prediction.\\n");
    }
```

```
}

void searchRecord() {
    char target[50];
    printf("Enter Item Name to search: ");
    scanf("%49s", target);

    int found = 0;
    for (int i = 0; i < count; i++) {
        if (strcmp(records[i].itemName, target) == 0) {
            printf("Found: Item: %s | Date: %s | Prepared: %d | Wasted: %d\\n",
                   records[i].itemName, records[i].date,
                   records[i].prepared, records[i].wasted);
            found = 1;
        }
    }

    if (!found) {
        printf("Record not found!\\n");
    }
}

void sortRecords() {
    struct FoodRecord temp;
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (records[j].wasted < records[j + 1].wasted) {
                temp = records[j];
                records[j] = records[j + 1];
                records[j + 1] = temp;
            }
        }
    }
    printf("Records sorted by highest waste successfully!\\n");
}

void saveToFile() {
    FILE *file = fopen("canteen_data.txt", "w");
    if (!file) {
        printf("Error opening file for saving!\\n");
        return;
    }

    for (int i = 0; i < count; i++) {
        fprintf(file, "%s %s %d %d %d %f\\n",
                records[i].itemName, records[i].date,
                records[i].prepared, records[i].consumed,
                records[i].wasted, records[i].wastePercentage);
    }

    fclose(file);
    printf("Data saved to file successfully!\\n");
}

void loadFromFile() {
    FILE *file = fopen("canteen_data.txt", "r");
    if (!file) return;

    count = 0;
    while (count < MAX &&
           fscanf(file, "%49s %14s %d %d %d %f",
                  records[count].itemName, records[count].date,
                  &records[count].prepared, &records[count].consumed,
                  &records[count].wasted,
                  &records[count].wastePercentage) == 6) {
        count++;
    }

    fclose(file);
}

int main() {
    loadFromFile();

    int choice;
    char queryName[50];

    while (1) {
        printf("\\n--- CANTEEN FOOD WASTE TRACKER ---\\n");
        printf("1. Add Record\\n");
        printf("2. Show Report\\n");
        printf("3. Sort by Waste (Highest First)\\n");
        printf("4. Search Record by Item Name\\n");
        printf("5. Predict Next Day Quantity\\n");
        printf("6. Save & Exit\\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input!\\n");
            break;
```

```
        }

        switch (choice) {
            case 1: addRecord(); break;
            case 2: showReport(); break;
            case 3: sortRecords(); break;
            case 4: searchRecord(); break;
            case 5:
                printf("Enter item name for prediction: ");
                scanf("%49s", queryName);
                predictQuantity(queryName);
                break;
            case 6:
                saveToFile();
                exit(0);
            default:
                printf("Invalid choice! Try again.\\n");
        }
    }
    return 0;
}
```